

AI-Powered Resume Screening System

Complete Final Viva Preparation Guide

Project: AI in Recruitment Process: A Study on the Use of AI for Resume Analysis

Team: Rohan Garg, Punit Sukhani, Saksham | Guide: Mrs. Gull Kaur



Delhi Technological University

1. One-Minute Project Summary

What the project does: The system takes one job description and multiple resumes, extracts readable text from every file, detects skills and years of experience, computes a weighted match score, explains the score in recruiter-friendly language, and displays ranked candidates with charts and score breakdowns.

Core idea: The project is a lightweight, explainable first-pass resume screening prototype. It deliberately uses Regex + TF-IDF + custom mathematical rules instead of heavy LLMs so that it remains fast, low-cost, transparent, and suitable for SME-style recruitment workflows.

How to say this in viva: Our system is not designed to replace HR. It is a decision-support tool that reduces the first screening workload while keeping the final judgement with the recruiter.

2. Problem Statement and Motivation

Modern job postings receive a large number of resumes. Manual screening is slow, inconsistent, and prone to bias. Traditional Applicant Tracking Systems rely heavily on exact keywords, which can miss good candidates who use different wording. On the other side, LLM-based systems understand semantics better but are expensive, slower, and less explainable. This project solves the middle problem: fast and explainable screening with enough intelligence for first-pass filtering.

- **Manual screening issue:** high time cost, fatigue, inconsistent judgement.
- **Keyword ATS issue:** fails on synonyms such as REST vs RESTful and can be tricked by stuffing.
- **LLM issue:** higher latency, memory/cost, and weaker auditability.
- **Our position:** use lightweight AI as a first-pass filter, not a final hiring authority.

3. Inputs, Outputs and User Flow

Inputs: recruiter login, job description text, one or more resumes in PDF, DOCX, TXT, PNG, JPG/JPEG format, and scoring weights configured through the upload dashboard.

Outputs: ranked candidate list, final match scores, matched and missing skills, detected soft skills, years-of-experience status, extra skills bonus, charts, and plain-English XAI explanation.

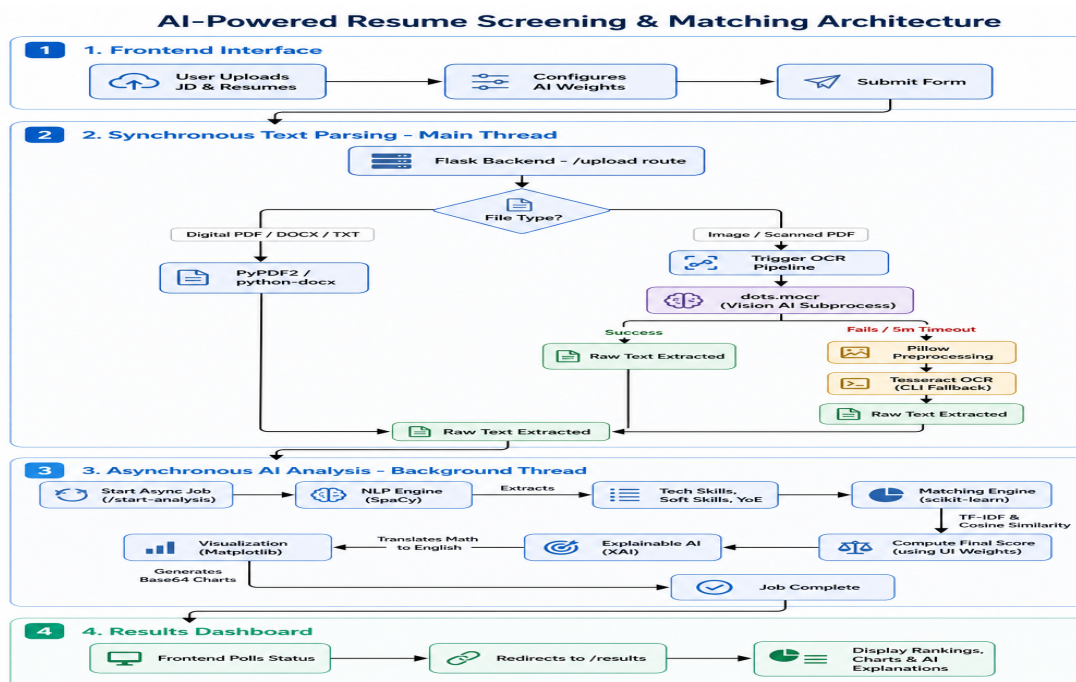


Figure: End-to-end architecture from frontend upload to parsing, NLP, matching, XAI, visualization, and final dashboard.

Step-wise flow from input to output

- 1 Recruiter logs in and opens the upload page.

- 2 Recruiter pastes the job description and uploads resumes.
- 3 Recruiter adjusts scoring weights: technical skills, experience, context match, soft skills, and bonus cap.
- 4 Backend validates files and stores uploaded data in server-side memory using a UUID key.
- 5 Analyze page starts a background analysis job and polls status every two seconds.
- 6 Parser extracts text digitally or routes scanned/image files to OCR.
- 7 NLP module extracts technical skills, soft skills, and years of experience.
- 8 Matching engine computes component scores and final weighted score.
- 9 Explainer converts the score dictionary into audit-ready text.
- 10 Dashboard displays ranked candidates, charts, skill tags, and explanation panels.

4. System Components and Responsibilities

Layer	Technology	Why it is used
Backend	Python, Flask	Lightweight REST-style web backend; easy to split parsing, NLP, matching, XAI and dashboard modules.
Frontend	HTML5, Bootstrap 5, Jinja2	Responsive pages with server-rendered score breakdowns and explanation panels.
Database/Auth	SQLite, SQLAlchemy, Flask-Login, Authlib	Simple local persistence, login protection, optional Google OAuth SSO.
Parsing	PyPDF2, python-docx, Pillow, pdf2image, Tesseract	Extracts text from digital files and OCRs scanned/image resumes.
NLP	SpaCy + Regex	Tokenization, entities, strict skill extraction, and YoE extraction.
Matching	scikit-learn TF-IDF + Cosine Similarity	Fast statistical context matching without heavy transformer models.
XAI/Charts	Custom Explainer, Matplotlib, Chart.js	Human-readable explanations and visual score dashboards.

Major source-code modules

- **app.py:** Creates the Flask application and registers blueprints.
- **models.py:** SQLAlchemy user model with email, name, password hash, SSO provider and avatar URL.
- **resume_parser/parser.py:** Handles PDF, DOCX, TXT, PNG, JPG/JPEG extraction and OCR fallback.
- **nlp_processing/processor.py:** Runs SpaCy, skill extraction, soft-skill extraction and YoE detection.
- **nlp_processing/skills_db.py:** Curated technical and soft-skill lists.
- **feature_extraction/extractor.py:** TF-IDF vectorization utilities.
- **matching_engine/matcher.py:** Weighted score computation and component contributions.
- **explainable_ai/explainer.py:** Converts score dictionaries into plain-English explanations.
- **visualization/charts.py:** Produces base64 Matplotlib charts embedded in HTML.
- **web_app/routes.py:** Upload, analyze, polling and results routes.
- **web_app/auth.py:** Email/password login and optional Google OAuth SSO.

5. Parsing and OCR Pipeline

Every file must become plain text before any AI step can run. The parser first attempts the cheapest reliable digital extraction. If text extraction fails or returns fewer than 50 characters, the document is treated as scanned or parser-resistant and routed to OCR. This design reduces format bias because a scanned or image-based resume still gets processed instead of being rejected.

Format	Primary method	Fallback / notes
.txt	Direct UTF-8 read	Uses errors='ignore' to avoid decode crashes.
.docx	python-docx paragraphs	Reads document XML text.
.pdf	PyPDF2 page extraction	If text length < 50, convert to image and OCR.
.png/.jpg	Pillow + OCR	Direct OCR path with preprocessing.

OCR preprocessing: RGBA flattening, grayscale conversion, contrast boost, sharpening, adaptive binarization, and Tesseract CLI extraction. The project uses direct Tesseract subprocess instead of pytesseract to avoid wrapper-level decode issues.

Algorithm: OCR-aware parsing

```
FUNCTION ParseDocument(file, extension):
  IF extension == ".txt": return read_text(file)
  IF extension == ".docx": return read_docx(file)
  IF extension == ".pdf":
    text = PyPDF2_extract(file)
    IF length(text) < 50:
      image_pages = convert_pdf_to_300dpi_images(file)
      return OCR_Pipeline(image_pages)
    return text
  IF extension in [".png", ".jpg", ".jpeg"]:
    return OCR_Pipeline(file)

FUNCTION OCR_Pipeline(image):
  flatten_transparency_to_white()
  grayscale()
  boost_contrast_and_sharpen()
  binarize_pixels(threshold=140)
  return tesseract_cli_extract_text()
```

6. NLP Processing and Regex Skill Extraction

The NLP layer uses SpaCy's en_core_web_sm model for tokenization, lemmatization and named entities. However, technical and soft skills are not trusted to generic NER. They are extracted using strict regex word-boundary matching against curated skill lists. This is deliberate: it prevents false matches such as the skill C appearing inside the word communication.

```
pattern = r'\b' + re.escape(skill) + r'\b'
```

YoE extraction follows a three-tier cascade: direct phrases such as '5+ years of experience', reversed phrases such as 'Experience: 5 years', and date math from ranges such as 'June 2024 - August 2025'. Values above 40 years are ignored, and long education-like date spans are filtered to avoid counting B.Tech duration as professional experience.

7. TF-IDF Context Matching

The system uses scikit-learn's TfidfVectorizer with max_features=5000, English stop-word removal and ngram_range=(1,2). Bigrams allow phrases such as machine learning or natural language to be represented better than plain unigrams. TF-IDF gives higher weight to distinctive terms and lower weight to common words.

Cosine similarity: Context Score = (Resume Vector dot JD Vector) / (||Resume Vector|| × ||JD Vector||). Cosine similarity is used because resumes may differ in length. It measures direction/topic alignment rather than raw length.

8. Matching and Scoring Formula

The final score combines hard requirements, soft evidence and contextual similarity. The first four weights always sum to 1.0, while the bonus is a capped additive offset.

Component	How it is computed	Why it matters
Tech Score	Matched required tech skills / required tech skills	Hard technical fit.
Soft Score	Matched requested soft skills / requested soft skills	Communication, teamwork, leadership evidence.
YoE Score	$\min(\text{resume_yoe} / \text{jd_yoe}, 1.0)$	Caps over-qualified candidates at 100%.
Context Score	TF-IDF cosine similarity	Broader textual alignment between JD and resume.
Bonus	$\min(\text{extra_tech} \times \text{bonus_step}, \text{bonus_cap})$	Rewards adjacent skills without hiding missing requirements.

```
Final Score =
(Tech Score × W_tech)
+ (Soft Score × W_soft)
+ (YoE Score × W_yoe)
+ (Context Score × W_context)
+ Extra Skills Bonus
```

Default weights:

Tech = 60%, YoE = 15%, Context = 15%, Soft = 10%, Bonus cap = 5%

9. Explainable AI Layer

The XAI layer is deterministic. It does not approximate a black-box model; it reads the exact score dictionary produced by the matcher and converts it into a recruiter-readable explanation. It reports final score, match band, experience status, matched required skills, missing skills, bonus skills, soft skills and score component contributions.

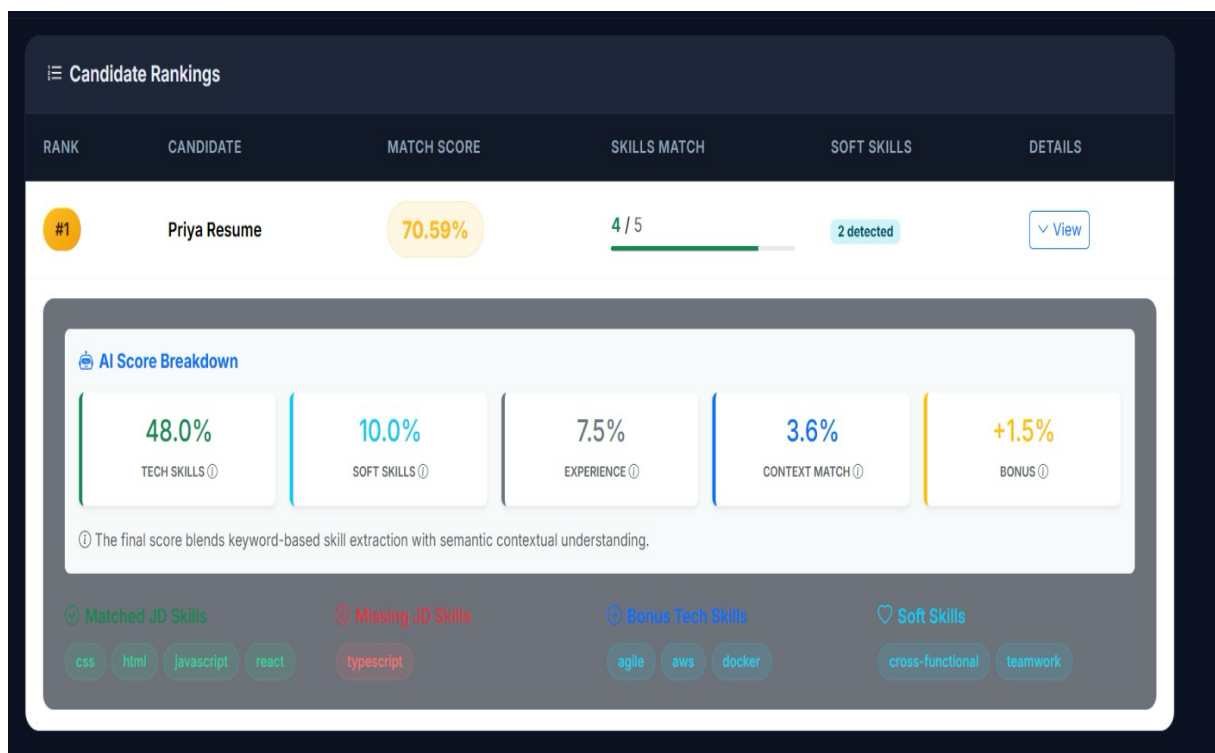


Figure: XAI score breakdown showing component contributions, matched/missing skills and bonus skills.

Candidate buckets: Strong match $\geq 75\%$, Moderate match $\geq 50\%$, Partial match $\geq 30\%$, Weak match $< 30\%$.

10. Frontend and Async Backend

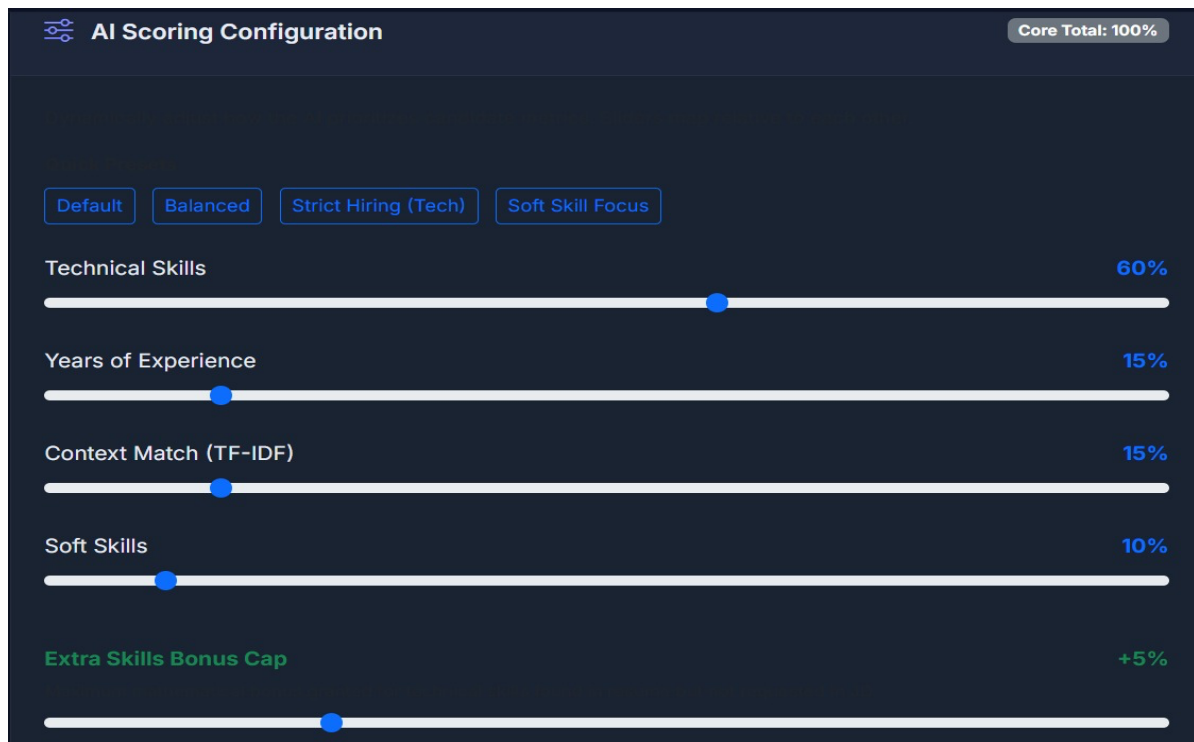


Figure: AI scoring configuration with recruiter-controlled weights and separate bonus cap.

The upload page contains five sliders: `weight_tech`, `weight_yoe`, `weight_context`, `weight_soft` and `weight_bonus`. The first four share a strict 100% pool. The JavaScript `normalizeSliders()` engine redistributes weights so the core total never exceeds 100%. The bonus slider is isolated because it is an additive cap, not part of the normalized core score.

The backend avoids blocking the main Flask request. Uploaded data is stored in `_data_store` using a UUID key. The analyze page calls `/start-analysis`, which creates a daemon background thread. The frontend polls `/analysis-status/` every two seconds and redirects to `/results` when complete.

```
POST /upload -> validate files, store data, redirect to /analyze
GET /analyze -> progress page
POST /start-analysis -> spawn Thread(_run_analysis), return job_id
GET /analysis-status/id -> return queued/loading/processing/done
GET /results -> render ranked candidates and charts
```

11. Benchmarks and What to Say in Viva

Benchmark	What was tested	Key result / viva point
1. Format Bias & OCR	TXT vs PDF vs PNG extraction and scoring.	PNG through OCR scored 90.5%, same as TXT. PDF scored 90.0% due to spacing loss.
2. Keyword Stuffing	Baseline, verbose/diluted resume, and keyword stuffer.	Regex kept hard-skill score stable; TF-IDF penalized dilution and contained stuffing.
3. XAI & Boundaries	Over-qualified YoE and extra skills bonus.	YoE capped at 100%; extra bonus +2.0% under 5.0% cap; explanation remained auditable.
4. A vs B vs C	Current Regex+TF-IDF vs Sentence-BERT vs future hybrid.	LLM understood semantics better; current model stayed strict and explainable.
5. Efficiency	100-resume stress test.	Approach A: 0.0062 sec, 0.12 MB; much faster/lighter than LLM alternatives.

Approach A vs B vs C

- **Approach A: Current app (Regex + TF-IDF).** Fast, lightweight, deterministic, explainable. Weakness: cannot understand all synonyms.
- **Approach C: Pure LLM/Sentence-BERT.** Better semantic understanding. Weakness: no strict hard boundaries, more latency and memory.
- **Approach B: Future hybrid.** Keep regex hard-skill shield and replace context score with LLM embeddings. Better semantics but slower than Approach A.

12. Strengths, Limitations and Future Work

Strengths	Limitations	Future Work
Fast CPU-friendly scoring; OCR fallback; explainable output; adjustable weights; keyword stuffing resistance.	Regex misses synonyms; TF-IDF lacks deep language meaning; skill ontology must be updated; OCR depends on scan quality; small controlled benchmark dataset.	Sentence-BERT hybrid context module; dynamic skill knowledge graph; fairness/bias audit; recruiter feedback loop; production database; larger real resume dataset.

13. Demo Script for Final Viva

- 1 Open the app and login/register.
- 2 Go to the upload screen and paste the job description.
- 3 Upload multiple resumes, ideally one TXT/PDF and one image/scanned sample.
- 4 Show the weight sliders and explain default weights: Tech 60, YoE 15, Context 15, Soft 10, Bonus cap 5.
- 5 Click submit and show the analyze/progress page.
- 6 Open results and explain ranking order, score breakdown, matched/missing skills and XAI text.
- 7 Mention why Approach A was chosen: fast, cheap, transparent first-pass screening.
- 8 End honestly: semantic models are better for synonyms, so future work is a hybrid model.

14. High-Probability Viva Questions

Question	Best answer
Why not use only LLM?	LLMs are semantically stronger but slower, costlier and less transparent. Our goal is first-pass filtering under hardware constraints.
Why regex for skills?	Strict word boundaries prevent false positives and make hard-skill extraction auditable.
Why TF-IDF?	It is fast, interpretable, CPU-friendly and good enough for lexical context matching.
Why cosine similarity?	It compares vector direction/topic alignment, not raw resume length.
What is XAI here?	A deterministic explanation layer that converts score components into recruiter-readable reasoning.
What is format bias?	A candidate can be penalized if the parser fails due to PDF/image formatting. OCR fallback reduces that risk.
Main limitation?	Regex and TF-IDF cannot fully understand synonyms or implicit skills.
Future work?	Hybrid Sentence-BERT context, dynamic skill ontology, fairness audit and recruiter feedback learning.

15. Detailed Algorithmic Flow to Explain to Ma'am

If asked for the complete algorithm, explain the project as a deterministic pipeline. The important point is that every candidate passes through the same stages, so comparison remains consistent.

INPUT:
 Job description J
 Resume batch $R = \{r_1, r_2, \dots, r_n\}$
 Recruiter weights $W = \{\text{tech}, \text{soft}, \text{yoe}, \text{context}\}$
 Extra skill bonus cap B_{max}

PROCESS:

1. Extract text from J.
2. Extract required tech skills, soft skills and YoE from J.
3. For each resume r_i :
 - a. Detect file type.
 - b. Extract text digitally if possible.
 - c. If extraction is weak, invoke OCR fallback.
 - d. Normalize extracted text.
 - e. Extract candidate skills, soft skills and YoE.
 - f. Compute tech, soft, YoE and TF-IDF context scores.
 - g. Compute capped extra skills bonus.
 - h. Calculate final weighted score.
 - i. Generate XAI explanation.
4. Sort candidates by final score.

OUTPUT:
 Ranked candidate list + score breakdown + XAI explanation + charts.

16. Route-by-Route Backend Walkthrough

Route / Page	What happens	Viva explanation
/upload	Receives JD, resumes and weights.	This is where input enters the AI pipeline.
/analyze	Shows progress page.	It prevents the user from waiting on a frozen upload page.
/start-analysis	Starts daemon background thread.	Heavy processing is detached from the request-response cycle.
/analysis-status/	Returns status and message.	Frontend polls it every two seconds.
/results	Renders ranked output.	Displays scores, charts, skills and explanations.

17. Internal Data Objects to Remember

- **NLP output:** tokens, lemmas, entities, technical_skills, soft_skills, years_of_experience.
- **Matcher output:** final_score, text_similarity, tech_skill_score, soft_skill_score, yoe_score, individual score contributions and bonus.
- **Explainer output:** candidate_name, match_score, matched_skills, missing_skills, extra_skills, detected_soft_skills and final explanation.
- **Job state:** queued, loading_models, processing_jd, processing_resumes, computing_scores, generating_explanations, creating_charts, done.

18. Why Each Design Choice Was Made

Choice	Reason	Trade-off
Flask	Lightweight and easy for modular prototype.	Not as batteries-included as Django.
SQLite	Zero-server local persistence.	Not ideal for very high concurrent writes.
Regex skills	Precise, auditable hard-skill matching.	Misses synonyms unless added to ontology.
TF-IDF	Fast, CPU-friendly context signal.	Less semantic than BERT/Sentence-BERT.
OCR fallback	Reduces format bias for scanned/image resumes.	Accuracy depends on image quality.
Async thread	Avoids blocking browser request during batch analysis.	In production, Celery/Redis would be stronger.

19. Last-Minute Revision Checklist

- Know the one-line project pitch.
- Be able to draw the flow: Upload -> Parse/OCR -> NLP -> TF-IDF -> Score -> XAI -> Dashboard.
- Remember default weights: Tech 60, YoE 15, Context 15, Soft 10, Bonus 5.
- Remember the main benchmark numbers: 90.5% TXT/PNG, 90.0% PDF, Approach A 0.0062 sec and 0.12 MB.
- Admit limitations confidently: regex misses synonyms, TF-IDF lacks deep semantics, OCR depends on scan quality.
- Close with future work: hybrid Sentence-BERT context, fairness audit, dynamic skill ontology and recruiter feedback loop.

20. Final Viva Closing Statement

This project demonstrates a practical AI recruitment decision-support system. It intentionally balances speed, cost, transparency and acceptable accuracy instead of chasing the heaviest model. The system is best understood as an explainable first-pass filter: it reduces HR workload, gives auditable reasons, handles multiple resume formats, and leaves the final decision to humans.